Software Development for Mobile Devices

Assignment 1 – Gymnastics App

Student Name: Do Dinh Bao Chung

Student ID: 105555822

Unit: COS30017

Lecturer Name: Nguyen Dang Khoa

Date: 11/10/2026

1. Introduction

A local gymnastics club is looking for a simple mobile app to help them manage the scores during practice of floor routines. Manual score recording is subject to inconsistencies, especially when different parts of a routine are scored differently, and deductions are based on certain rules. The Gymnastics App aims to tackle these problems by offering a simple interface for recording performances and calculating scores based on the club's modified scoring system.

The application is designed on Android using Kotlin for scoring logic and event handling and XML for user interface designing. Each routine consists of 10 elements, divided into three scoring zones, each with a different value. Users can log completed elements, deduct where applicable, and reset the routine. The current score and number of the element enforcing scoring constraints to prevent invalid operations are displayed in the application.

This report describes the requirements analysis, the application design, the implementation process, the technical challenges, and the testing procedures. Special attention is paid to state management, handling of Activity lifecycle, screen orientation, and localization, as these are important for consistent application behavior on different device configurations.

2. Requirement Analysis

2.1 Functional Requirements

The Gymnastics App is designed to manage a floor routine, consisting of ten elements in a row. It is primarily used to record the elements that have been completed, calculate the scores, apply deductions and display the current status of the routine.

This application has three interactive buttons . Perform, Deduction, and Reset. Clicking on Perform will mark the current element as completed and increase the score by the assigned zone. If the scoring rules allow it, the Deduction button will penalize for two points. The Reset button will reset the application to its starting state.

The user interface must show the current score, the element number, and the application title. It also has to give the proper visual feedback as the routine is going on, and tell when all ten elements are done.

2.2 Scoring System

The scoring system divides the ten elements into three zones with different point values. The difficulty of the routine increases throughout and the maximum possible score is 20 points.

Zone	Elements	Plus Points	Maximum Point
Basic	1-3	1	3
Intermediate	4-7	2	8
Advanced	8-10	3	9
Total	10		20

Table 1: ***Scoring zones and point allocation***

The application will decide the correct point value depending on the element being performed. The score begins at zero and must be between 0 and 20 points at all times.

Each zone is visually identified by a specific color: blue for Basic, green for Intermediate, and orange for Advanced. It gives you instant feedback on where you are in the routine.

2.3 Functional Requirements

Besides the score calculation, the application has to impose several restrictions in an effort to avoid invalid scoring operations.

ID	Constraint	Expected Behaviour
01	Initial state	Score is 0 and the current element is 1
02	Sequential progression	Elements are performed in order without backtracking
03	Deduction egibility	Deduction is unavailable before the first completed element
04	Deduction value	A valid deduction subtracts 2 points, without allowing a negative score
05	Deduction limit	Only one deduction is permitted per routine

06	Performance Restriction	No additional elements can be performed after a deduction
07	Routine Completion	After element 10, further performances and deductions are disabled.
08	Reset Operation	Reset restores the initial score, element, and deduction state

Table 2: ***Scoring constraints***

These constraints define 3 main application states: active, deducted and completed. Users can perform elements and take an eligible deduction during the active state. Applying a deduction stops further progression, and the tenth element rounds off the routine. Either terminal state allows a new routine to begin through the Reset operation.

This state based interpretation is important as button availability depends on the current score as well as the progression and deduction history .

2.4 Functional Requirements

The application should offer usable interfaces for both portrait and landscape orientations. These configurations necessitate different XML layout structures while maintaining equivalent functionality and consistent interaction behavior.

The app also needs to remember its score when the device is rotated. This means that when the Android system recreates the Activity, it retains the current score, the number of elements that have been completed, and the deduction status.

The app should also support an additional language via Android string resources and include sensible logging that can be used to help with debugging and verification.

These requirements set out the functional and technical scope of the application. The following sections describe how the requirements are realized using Kotlin and XML and how their correctness is verified through systematic testing.

ID	Requirement	Verification Criterion
01	Poitrait and landscape layouts	Both orientations display the required controls correctly

02	State preservation	Score, progression and deduction state remain unchanged after rotation
03	Localization	Interface text is available in an additional language
04	Logging	Relevant user actions and state changes are recorded in Logcat

Table 3: ***Technical requirements and verification criteria***

3. Design and Implementation

3.1 UI Design

The Gymnastics App uses a single-Activity architecture with MainActivity.kt to handle application logic and XML resources that define the user interface. This is a good structure for a small scoring application because all the interactions are on one screen and no extra navigation is required.

The interface shows the application title, score display, element indicator, routine status information and three buttons: Perform, Deduction and Reset. These components give the information and control needed to handle a gymnastics routine.

Two layout resources are designed to accommodate different screen orientations:

- Portrait Layout: res/layout/activity_main.xml uses a vertical LinearLayout to arrange interface components sequentially. (as shown in Figure 1)

- Landscape Layout: res/layout-land/activity_main.xml uses a RelativeLayout to organize components according to their relative positions and make better use of horizontal screen space. (as shown in Figure 2)

Android automatically selects the correct layout resource based on the device configuration. The view IDs are the same for both layouts, so MainActivity.kt so that it can refer to the same interface components regardless of orientation. This way you don't have to duplicate any application logic, and you get consistent functionality in both layouts.

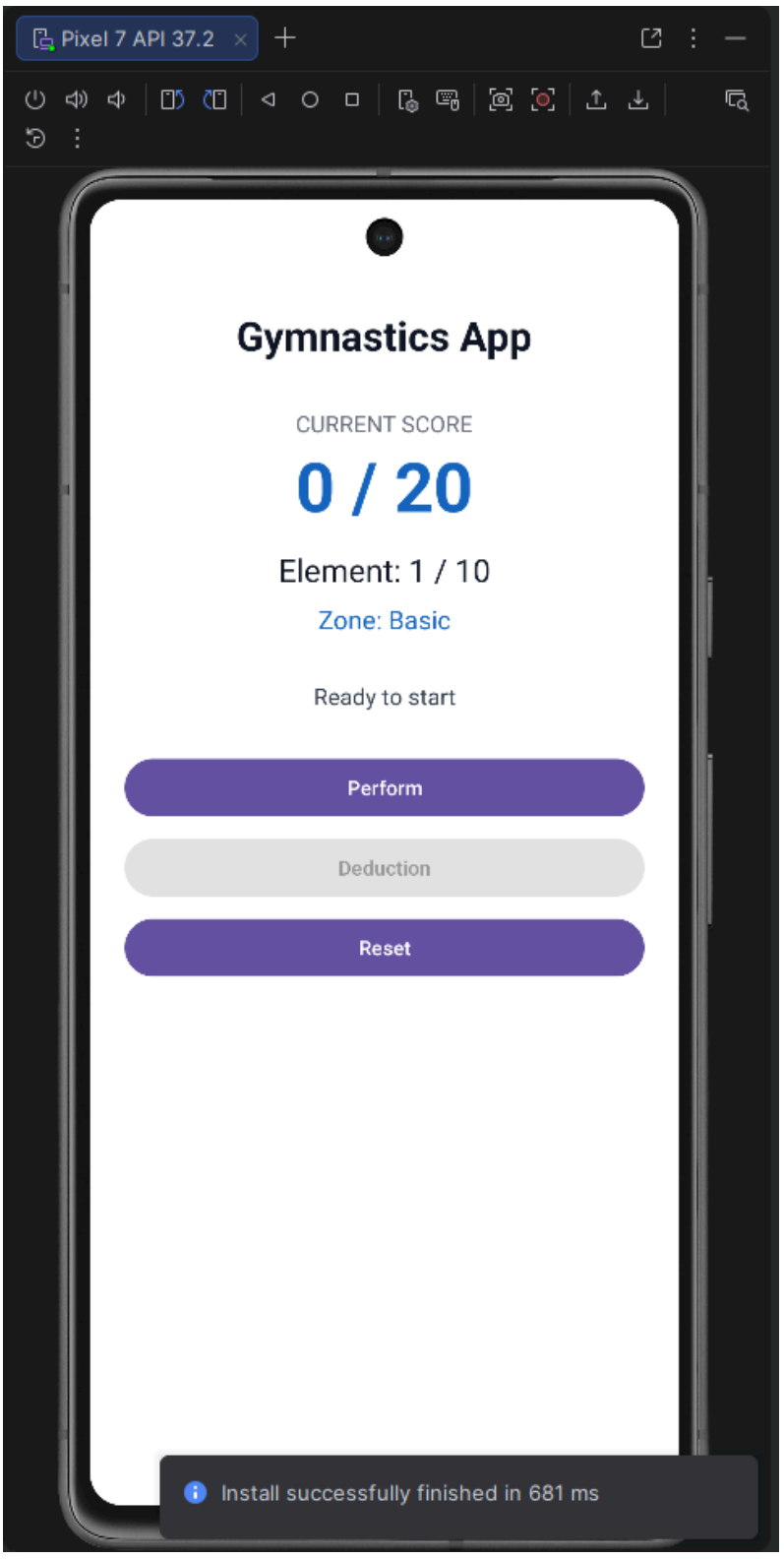


Figure 1: ***Portrait Orientation***

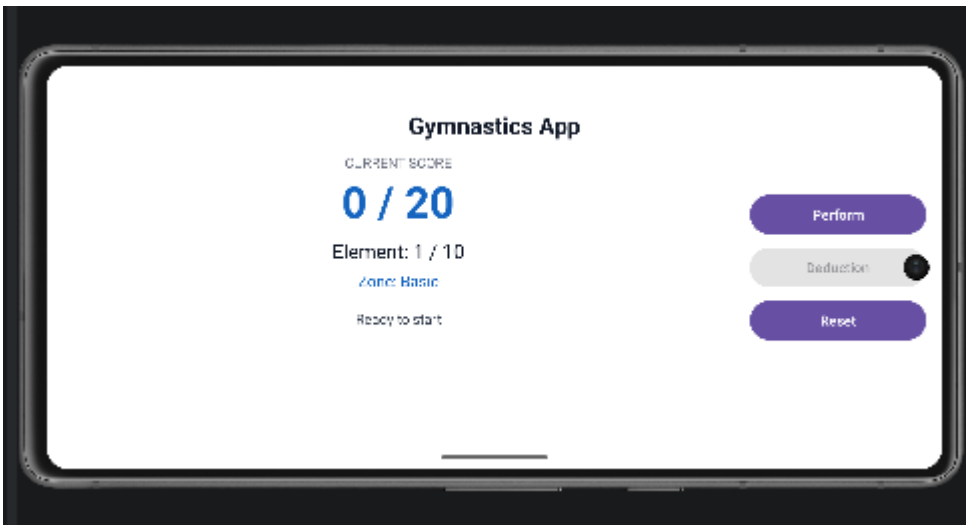


Figure 2: ***Landscape Orientation***

3.2 Application State Management

The application (shown in figure 3a) uses three primary Kotlin variables to represent its scoring state:

Variable	Purpose
score	Store the accumulated score
completedElements	Tracks the number of successfully performed elements
deductionTaken	Indicates whether a deduction has already been applied

```
24      private var score = 0
25      private var completedElements = 0
26      private var deductionTaken = false
```

Figure 3a: ***Primary state variables used to manage routine progression.***

The application does not treat the current element as a separate, independent variable. Instead it derives the current element from completedElements. The next element is calculated as the number of completed elements + 1. The displayed value is clamped to the max of ten with coerceAtMost(TOTAL_ELEMENTS).

This design eliminates redundant state and the possibility of inconsistency between routine progression and the element number displayed. It also makes it easy to reset and recover an application, because the interface can be rebuilt from a small set of primary state variables.

Furthermore, deductionTaken tells us something that the score itself does not. For example, zero could mean that a routine was not performed or that a routine was stopped due to a deduction. By keeping this Boolean state, the application can distinguish these situations and apply the corresponding restrictions.

```
private fun renderState() {
    val currentElement =
        (completedElements + 1).coerceAtMost( maximumValue = TOTAL_ELEMENTS)
```

Figure 3b: ***Deduction State Management and Routine Progression Control.***

3.3 Event Handling and Scoring Logic

The user is enabled to interact with the three buttons by Kotlin click listeners. Each listener checks the current state of the application before modifying the respective variables.

Perform Button

If you choose Perform, the application first checks to see if the routine is still active and fewer than 10 elements have been completed. If the operation is valid, the points corresponding to the current element are awarded:

- Elements 1-3: 1 point per element.
- Elements 4, 5, 6, and 7: 2 points each.
- Elements 8-10: 3 points apiece.

The relevant points are added and then completedElements is incremented. The routine enters the completed state when the tenth element is executed and performs no further scoring operations.

```
private fun calculatePoints(element: Int): Int {
    return when (element) {
        in 1 ≤ .. ≤ 3 -> 1
        in 4 ≤ .. ≤ 7 -> 2
        in 8 ≤ .. ≤ 10 -> 3
        else -> 0
    }
}

private fun performElement() {
    Log.d(TAG, msg = "Perform button clicked")

    if (deductionTaken || completedElements >= TOTAL_ELEMENTS) {
        Log.w(TAG, msg = "Perform blocked")
        return
    }

    val nextElement = completedElements + 1
    val points = calculatePoints(nextElement)

    score = (score + points).coerceIn(0, MAX_SCORE)
    completedElements = nextElement

    Log.d(
        TAG,
        msg = "Performed element=$nextElement, " +
                "added=$points, score=$score"
    )

    renderState()
}
```

Figure 4a: ***Element-Based Scoring and Perform Action Validation.***

Deduction Button

The applyDeduction() function checks to see if the deduction is allowed before changing the score. The routine is finished . The score is zero . A previous deduction is taken . No element is completed . A deduction is disallowed .

If deduction is applicable : score -= 2 . Use coerceAtLeast(0) to avoid a negative score . Then the DeductionTaken flag is set to true and no further Perform and Deduction operations are allowed.

```kotlin
private fun applyDeduction() {
    Log.d(TAG, msg = "Deduction button clicked")

    if (completedElements == 0 ||
        completedElements >= TOTAL_ELEMENTS ||
        score == 0
    ) {
        Log.w(TAG, msg = "Deduction blocked")
        return
    }

    score = (score - 2).coerceAtLeast( minimumValue = 0)
    deductionTaken = true

    Log.d(TAG, msg = "Deduction applied, score=$score")

    renderState()
}
```

Figure 4b: ***Deduction Validation and Score Boundary Enforcement.***

Reset Button

The resetRoutine() function resets to the initial state, setting score and completedElements to zero and deductionTaken to false. Then the interface is refreshed and a new routine can start without restarting the Activity.

Combined these functions implement a consistent sequence of processing: validating an operation, updating the relevant state variables and refreshing the interface.

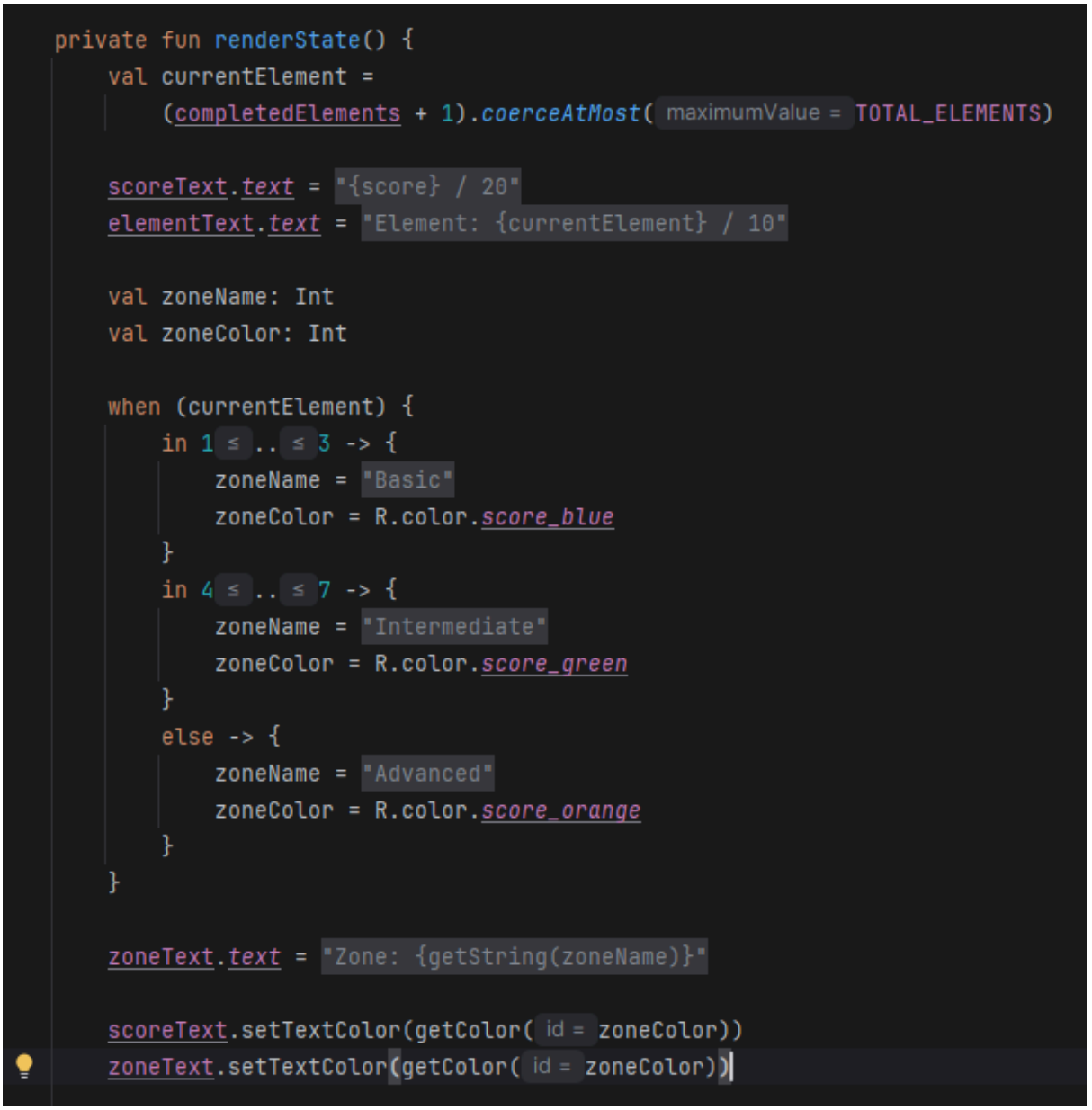
3.4 Centralized UI Rendering

Can be seen from figure 5a, the application has a centralized renderState() function . This function is used to synchronize application's visual components with underlying Kotlin state .

Instead of changing state of several interface elements separately in each button handler, the state-changing operations call renderState() after changing the variables which need to be changed. This decouples scoring logic from interface presentation and reduces duplicated UI update code.

The function calculates the displayed element and updates the score and element indicators using localized string resources. It then uses a when

expression to pick the correct zone and then sets the appropriate color for scoreText and zoneText.



```
private fun renderState() {
    val currentElement =
        (completedElements + 1).coerceAtMost( maximumValue = TOTAL_ELEMENTS)

    scoreText.text = "{score} / 20"
    elementText.text = "Element: {currentElement} / 10"

    val zoneName: Int
    val zoneColor: Int

    when (currentElement) {
        in 1 ≤ .. ≤ 3 -> {
            zoneName = "Basic"
            zoneColor = R.color.score_blue
        }
        in 4 ≤ .. ≤ 7 -> {
            zoneName = "Intermediate"
            zoneColor = R.color.score_green
        }
        else -> {
            zoneName = "Advanced"
            zoneColor = R.color.score_orange
        }
    }

    zoneText.text = "Zone: {getString(zoneName)}"

    scoreText.setTextColor(getColor( id = zoneColor))
    zoneText.setTextColor(getColor( id = zoneColor))
```

Figure 5a: ***Score display, element progression, and dynamic scoring-zone rendering.***

In figure 5b, renderState() also decides whether the buttons are enabled or disabled depending on the current state of the application. After subtracting or completing the 10th item, the Perform button is disabled. The Deduction button is only enabled when an eligible element has been completed, the score is still positive and a deduction has not been taken previously.

The function also updates the statusText to communicate if the routine is ready, in progress, terminated by deduction or complete.

```
    performButton.isEnabled =
        !deductionTaken && completedElements < TOTAL_ELEMENTS

    deductionButton.isEnabled =
        completedElements in 1 ≤ until < TOTAL_ELEMENTS && score > 0

    statusText.text = when {
        completedElements == TOTAL_ELEMENTS ->
            "Routine complete! Final score: {score} / 20"
        deductionTaken ->
            "Deduction applied. No more performances."
        completedElements == 0 ->
            "Ready to start"
        else ->
            getString( resId = R.string.status_progress)
    }
}
```

Figure 5b: State-dependent button availability and routine status updates.

By centralizing these responsibilities we ensure that the interface is always in sync with the current scoring state. The same function is called at the start of the Activity so that the interface can be rebuilt when the saved state has been restored.

4. Technical Challenges

4.1. Activity State Saving and Lifecycle

One of the major technical challenges in Android development is preserving application state when an Activity is recreated. Rotation of the device can cause the configuration to change. In this case, Android will destroy and recreate the Activity. The regular Kotlin instance variables are re-initialized upon re-creation, and without a proper state restoration mechanism, the current score, completed elements and deduction status could be lost.

To solve this problem, the application overrides onSaveInstanceState() to save three important variables in a Bundle: score, completedElements and deductionTaken. A key is used to store each value and savedInstanceState is used in onCreate() to retrieve the values. (can be seen in figure 6)

```
if (savedInstanceState != null) {
    score = savedInstanceState.getInt(
        KEY_SCORE, defaultValue = 0
    ).coerceIn(0, MAX_SCORE)

    completedElements = savedInstanceState.getInt(
        KEY_COMPLETED, defaultValue = 0
    ).coerceIn(0, TOTAL_ELEMENTS)

    deductionTaken = savedInstanceState.getBoolean(
        KEY_DEDUCTION, defaultValue = false
    )
```

```
override fun onSaveInstanceState(outState: Bundle) {
    outState.putInt(KEY_SCORE, score)
    outState.putInt(KEY_COMPLETED, completedElements)
    outState.putBoolean(KEY_DEDUCTION, deductionTaken)

    Log.d(
        TAG,
        msg = "State saved: score=$score, " +
                "completed=$completedElements, " +
                "deduction=$deductionTaken"
    )

    super.onSaveInstanceState(outState)
}
```

Figure 6(both 2 pics): ***Saving and restoring scoring state using an Android Bundle.***

During restoration, coerceIn() limits the restored score and completed element count to their valid ranges. The application restores the variables, initializes the interface components, and calls renderState() to rebuild the displayed score, element indicator, and button states.

It decouples transient instance-state persistence from permanent data storage. A Bundle provides state recovery on Activity recreation, but not a permanent record of past routines. For the present assignment it is enough to keep the active routine with no addition of a database.

4.2. Adaptation of layout and orientation

Supporting portrait and landscape is tricky since the sizes of screens and component layout are different. In both configurations a single fixed layout may not be an optimal use of the available space.

This is handled by the application through separate XML layout resources in Figure 7. The portrait mode interface is a vertical LinearLayout. The landscape mode interface is a RelativeLayout with independent information and control panels.

Using the same IDs in both layouts , the Activity can just re - bind its UI references . No additional interaction logic is needed . This design, in combination with saved-state restoration, allows the scoring process to continue seamlessly when the device orientation changes.

Another implementation consideration is the android system bars. The applySystemBarInsets() function determines the space taken by the system bars and it sets the root layout's padding with WindowInsetsCompat. This reduces the possibility of interface elements clashing with system navigation or status elements.

```
    private fun applySystemBarInsets() {
        val root = findViewById<View>(R.id.rootLayout)

        val left = root.paddingLeft
        val top = root.paddingTop
        val right = root.paddingRight
        val bottom = root.paddingBottom

        ViewCompat.setOnApplyWindowInsetsListener( view = root) { view, insets ->
            val bars = insets.getInsets(
                typeMask = WindowInsetsCompat.Type.systemBars()
            )

            view.setPadding(
                left + bars.left,
                top + bars.top,
                right + bars.right,
                bottom + bars.bottom
            )

            insets
        }
    }
}
```

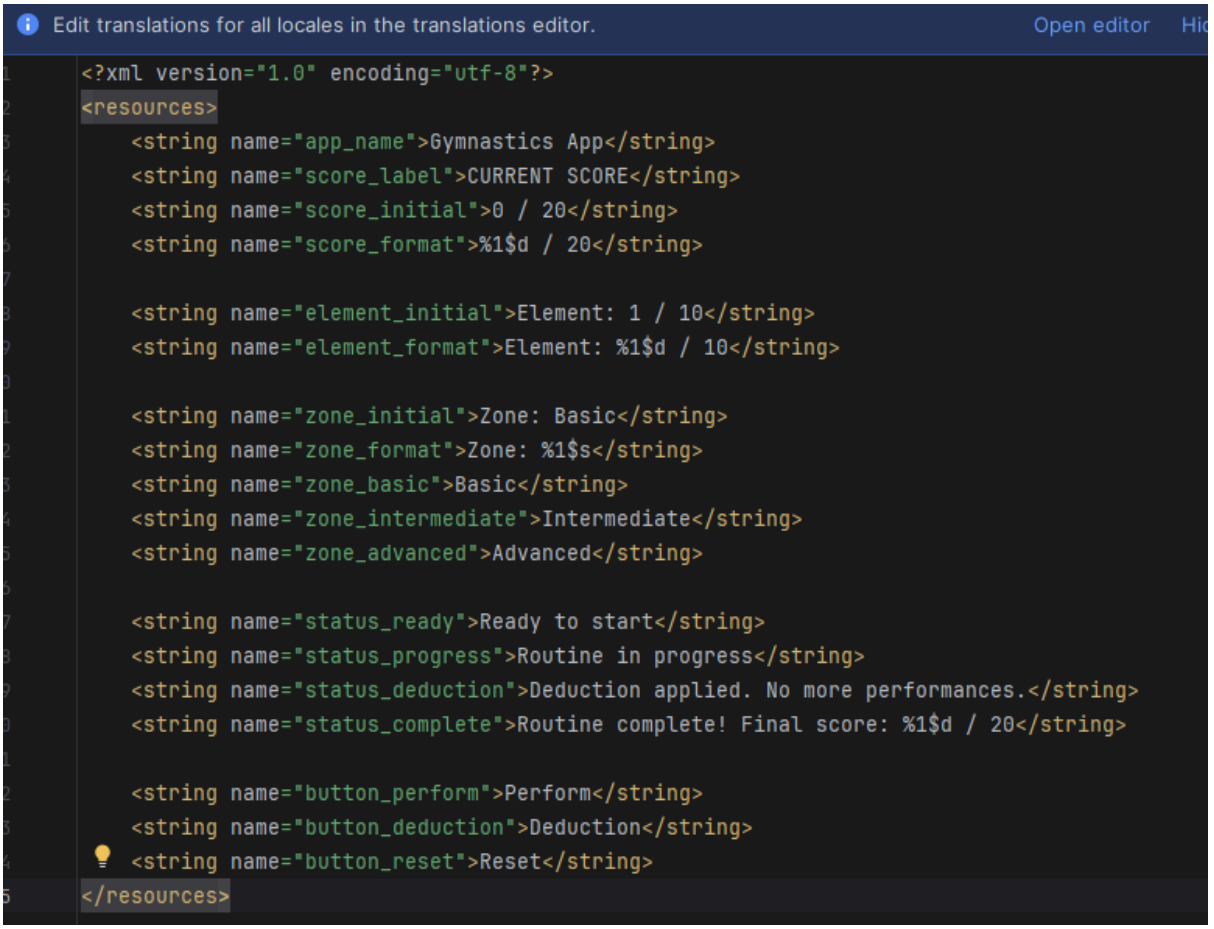
Figure 7: ***System Bar Insets Handling for Adaptive Layout Padding.***

Hence, the orientation-specific layout design and state restoration mechanism solve two inter-related problems of visual arrangement adaptation and underlying scoring information preservation.

4.3 Localization and String Resource Management

Localization adds another level of complexity. If you try to display text using hard-coded strings, it will make supporting multiple languages very difficult.

As in Figure 8, application defines re-usable entries in strings separating interface text from Kotlin logic.xml. English and Vietnamese versions of score labels, element indicators, scoring zones, buttons and routine status messages are provided.



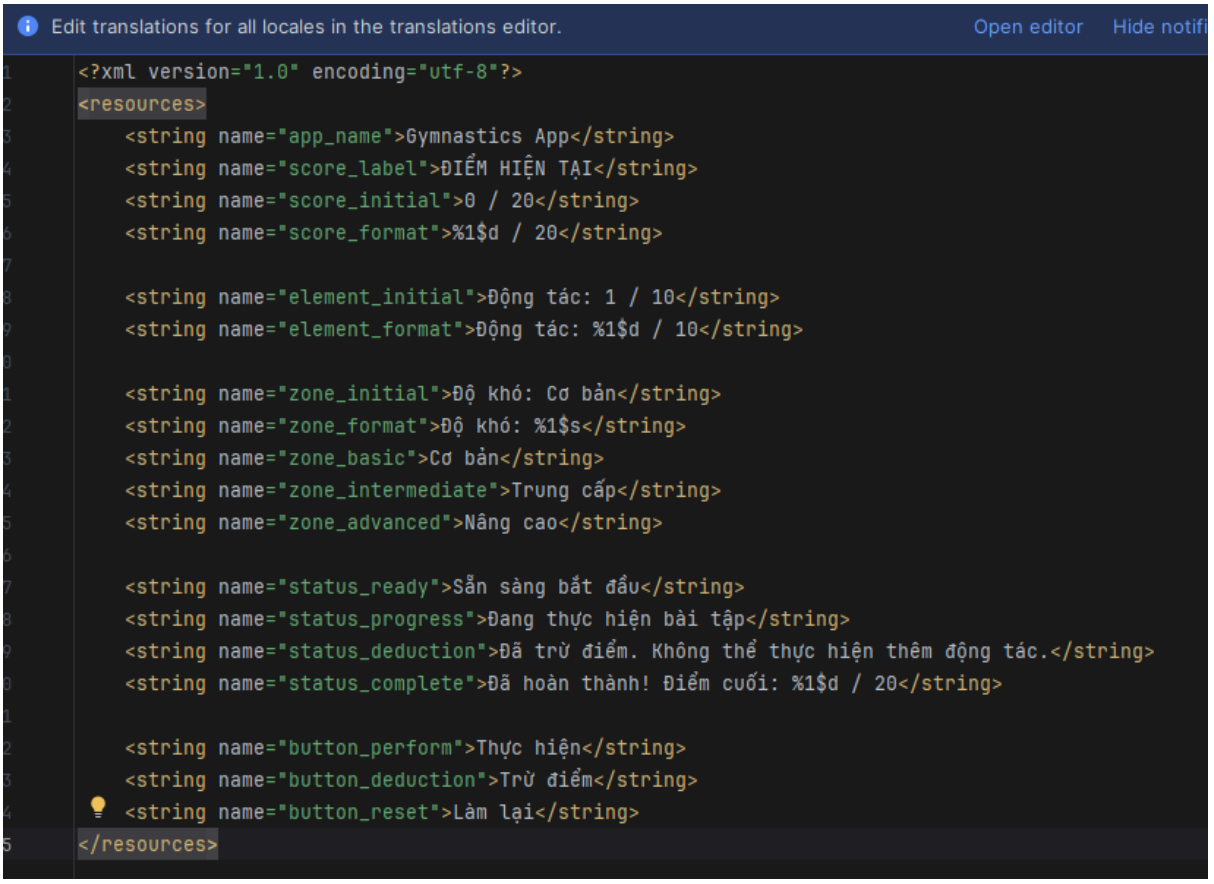


Figure 8(both 2 pics): ***English and Vietnamese String Resources for Interface Localization.***

In MainActivity.kt, dynamic messages are constructed using getString() with resource identifiers and formatting arguments. For example, the current score and element number are inserted into predefined string templates, not built with hard-coded English text.

Doing so allows Android to select the correct localized resources for the device's language, assuming that you put the translations into the right resource directories. It also separates presentation text from application logic, which makes it more maintainable and easier to add new languages in the future.

4.4 Debugging and State Validation

Debugging a state-dependent application means knowing if an unexpected result came from the scoring calculations, state transitions, or interface rendering.

The application uses Android's Log utility for diagnostics, using a consistent tag GymnasticsApp. Debug messages include button presses, successful

scoring operations, deductions, resets, and state-saving or restoration events. Invalid scoring operations are rejected and warning messages are generated.

```
if (deductionTaken || completedElements >= TOTAL_ELEMENTS) {
    Log.w(TAG, msg = "Perform blocked")
    return
}
```

Figure 9: ***Invalid action***

```
Log.d(
    TAG,
    msg = "Performed element=$nextElement, " +
          "added=$points, score=$score"
)
```

Figure 10: ***Successful action***

These logs show you the execution order and the values of important state variables. For example, a successful Perform operation logs the element processed, points awarded, and resulting score. Similarly, you can compare saved-state messages to restored-state messages when looking into Activity recreation.

Logging is particularly helpful since a displayed value by itself may not indicate whether the underlying state transition was processed correctly. The Logcat observations can be combined with functional testing to assess more systematically the application behavior.

5. Testing and Verification

5.1. Test approach

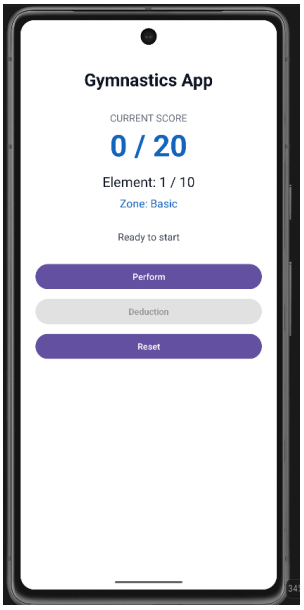
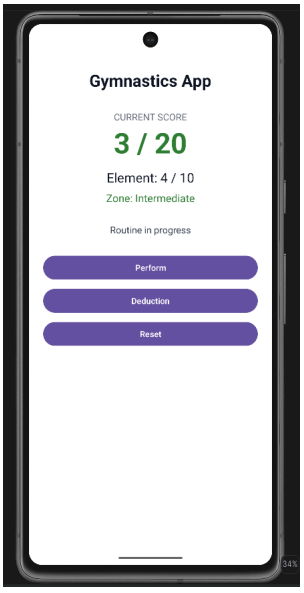
The implementation of the Gymnastics App is evaluated by functional and boundary-value testing to check whether it satisfies the requirements identified in Section 2. The testing process revolves around scoring accuracy, element progression, deduction restrictions, reset functionality and interface behavior on different Android configurations.

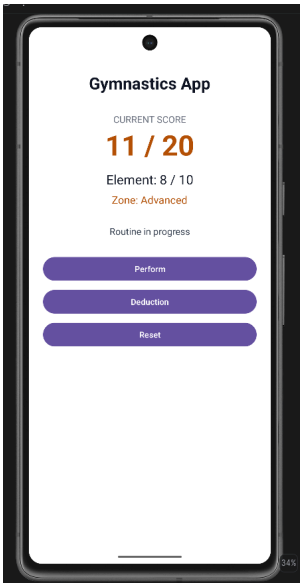
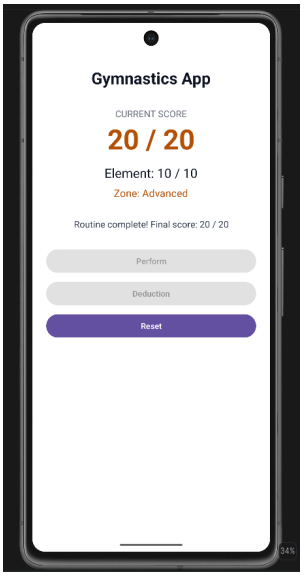
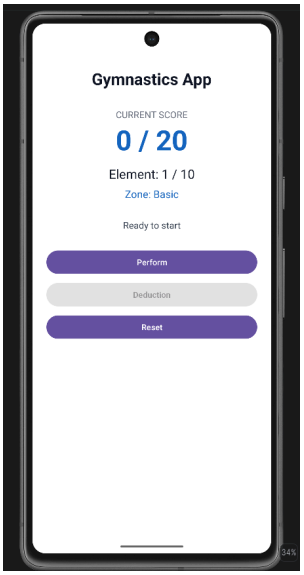
Test Scenarios are created to test normal operations and invalid user interactions. In particular, the boundaries between the scoring zones, the

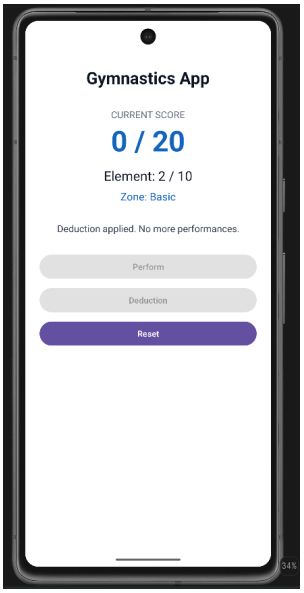
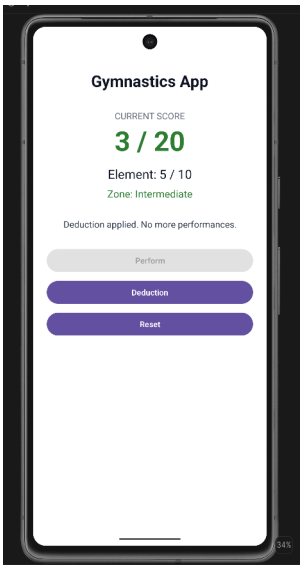
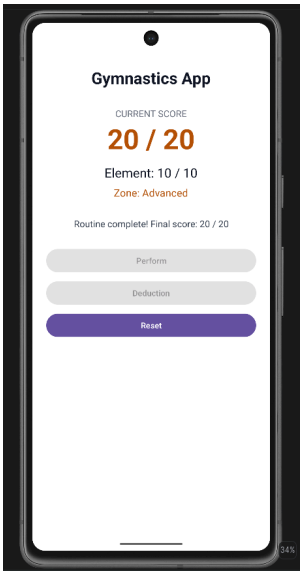
lower and upper limits of the score and limits set due to deductions or the end of the routine.

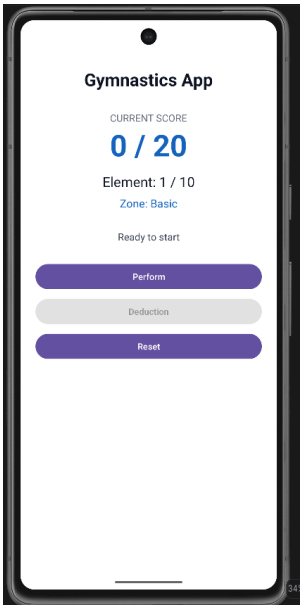
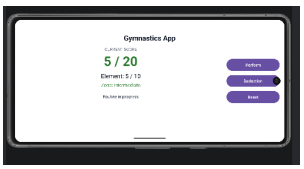
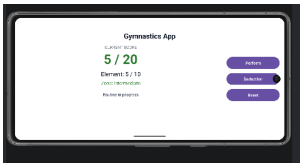
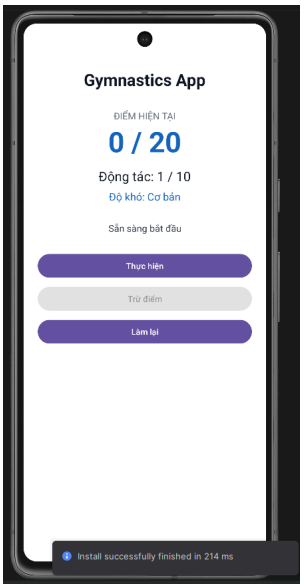
Additional verification includes portrait and landscape layout, Activity state restoration on device rotation, and English-Vietnamese localization. The observed behavior is documented with screenshots of the Android Emulator and Logcat output.

5.2. Functionality Test Cases

ID	Test Scenario	Expected Result	Result
1	Launch application	Score = 0; Element = 1; Basic zone	
2	Perform elements 1–3	Score = 3; next element = 4	

3	Perform elements 1–7	Score = 11; next element = 8	
4	Complete all elements	Score = 20; completion message displayed	
5	Attempt deduction before first element	Deduction unavailable; score remains 0	

6	Apply deduction after element 1	Score becomes 0; further performance disabled	
7	Apply deduction after four elements	Score decreases from 5 to 3;	
8	Attempt actions after completing element 10	Perform and Deduction disabled	

9	Reset after scoring or deduction	Initial score, element, and button states restored	
10	Rotate device after four elements	Score = 5 and progression preserved	
11	Switch portrait to landscape	Interface rearranged without losing required controls	
12	Change system language to Vietnamese	Available interface strings displayed in Vietnamese	

In Figure 11, Logcat output provides supplementary evidence by recording scoring operations, invalid actions, and Activity state-saving or restoration events.

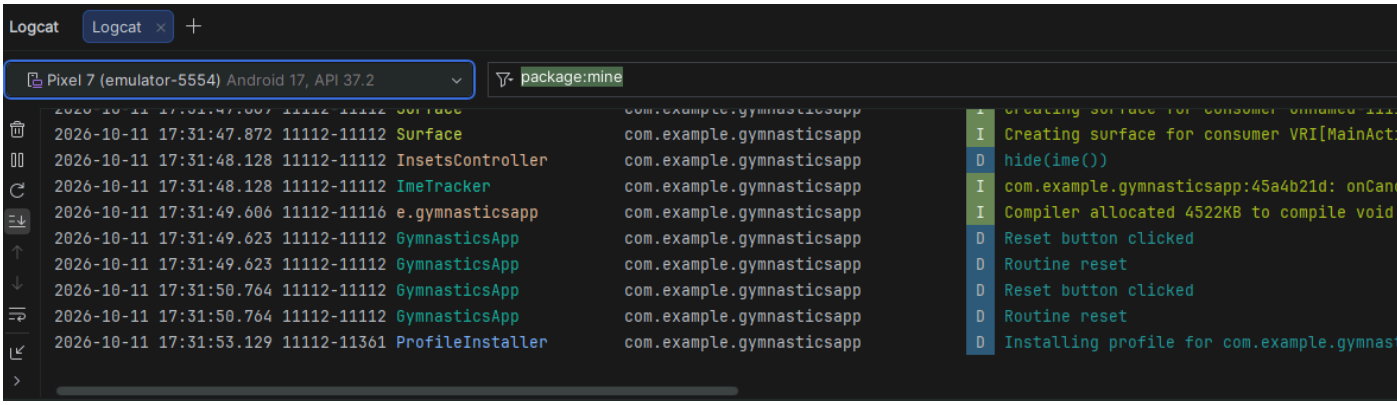


Figure 11: ***Logcat output showing scoring operations and Activity state restoration.***

5.3. Verification Criteria and Limitations

The test case is considered successful only if the observed application behavior coincides with the expected result. The Screenshots and Logcat records are a must for the relevant test cases to prove that the test case was executed.

Handling of deduction needs special attention, since the application needs to enforce restrictions through the underlying scoring logic and the enabled states of the interface controls. Thus, tests should include cases where the score remains positive after a deduction, as these can expose whether repeated deductions are incorrectly permitted.

The verification is limited to the selected emulator configurations and the manually executed scenarios. These tests can verify conformance to the functional requirements specified under the tested conditions, but they do not completely cover all Android devices or versions of the operating system.